

Test Documentation. Challenges API Tests

Overview

This document provides comprehensive documentation for the test suite covering the Challenges API endpoints. The suite contains **37 tests** organized into 10 logical groups, achieving near-complete coverage of the challenge-related functionality.

Test Categories

1. Challenge Creation Tests (**TestChallengeCreation**)

Test Name	Description
test_create_challenge_success	Verifies successful creation of a challenge with multiple exercises; ensures creator is auto-joined and receives a join code
test_create_challenge_with_weekly_schedule	Tests challenge creation with weekly scheduling (Monday, Wednesday, Friday); validates schedule_days array storage
test_create_challenge_with_open_end	Ensures challenges can be created without an end_date (open-ended challenges)
test_create_challenge_fails_without_exercises	Validates that challenges require at least one exercise (422 validation error)
test_create_challenge_fails_with_duplicate_exercises	Ensures the same exercise cannot be

Test Name	Description
	added twice to one challenge
<code>test_create_challenge_fails_with_invalid_exercise</code>	Tests handling of non-existent exercise_id (400 Bad Request)
<code>test_create_challenge_fails_with_invalid_schedule_days</code>	Validates schedule_days are in range 1-7 for weekly schedules
<code>test_create_challenge_fails_with_end_date_before_start</code>	Ensures end_date cannot precede start_date
<code>test_create_challenge_without_auth</code>	Verifies authentication is required to create challenges (401 Unauthorized)

2. Challenge Detail Tests (`TestChallengeDetail`)

Test Name	Description
<code>test_get_challenge_detail</code>	Validates retrieval of complete challenge details including exercises, participants count, and joined status
<code>test_get_challenge_detail_not_found</code>	Ensures 404 response for non-existent challenge IDs
<code>test_get_challenge_detail_join_code_only_for_creator</code>	Confirms join_code is visible only to the challenge creator, not to other users

3. Challenge Edit Tests (**TestChallengeEdit**)

Test Name	Description
test_edit_challenge_name	Verifies successful editing of name and description fields
test_edit_challenge_by_non_creator	Ensures only the creator can edit a challenge (403 Forbidden)
test_edit_challenge_not_found	Tests 404 response when attempting to edit non-existent challenge

4. Join/Leave Tests (**TestJoinLeave**)

Test Name	Description
test_join_by_code_success	Tests successful challenge joining using a valid join_code
test_join_by_code_invalid	Ensures 404 response for invalid join codes
test_join_already_joined	Validates users cannot join the same challenge twice (409 Conflict)
test_join_inactive_challenge	Ensures users cannot join archived/completed challenges (409 Conflict)
test_leave_challenge	Tests successful exit from a challenge; verifies participation is removed
test_leave_not_participant	Ensures 404 response when non-participant attempts to leave

5. Session Submission Tests (**TestSessionSubmission**) Core Logic

Test Name	Description
test_submit_session_closes_exercise	Verifies an exercise is marked closed when clean_reps reach or exceed goal

Test Name	Description
<code>test_submit_session_closes_day</code>	Tests full day completion when all exercises in a challenge are closed; validates streak increment
<code>test_submit_session_fails_if_not_active</code>	Ensures sessions cannot be submitted to archived challenges (409 Conflict)
<code>test_submit_session_fails_if_not_participant</code>	Validates only participants can submit sessions (403 Forbidden)
<code>test_submit_session_fails_with_invalid_exercise</code>	Ensures exercises must belong to the challenge (404 Not Found)
<code>test_submit_session_accumulates_today</code>	Tests multiple sessions per day accumulate; first session may not close, second does

6. Leaderboard Tests (`TestLeaderboard`)

Test Name	Description
<code>test_leaderboard_returns_sorted</code>	Validates leaderboard sorting by <code>days_completed</code> , <code>challenge_streak</code> , <code>total_clean_reps</code> , and <code>joined_at</code>
<code>test_leaderboard_requires_auth</code>	Ensures authentication is required for leaderboard access (401 Unauthorized)

7. Presets Tests (`TestPresets`)

Test Name	Description
<code>test_get_presets</code>	Verifies retrieval of preset/system challenges list; returns empty list or presets

8. Me Routes Tests (**TestMeRoutes**)

Test Name	Description
<code>test_me_profile</code>	Validates user profile endpoint returns correct user data including streak information
<code>test_my_challenges_active</code>	Filters and returns only active challenges for the current user
<code>test_my_challenges_archived</code>	Filters and returns only archived challenges for the current user

9. Archive Tests (**TestArchive**)

Test Name	Description
<code>test_archive_challenge_by_creator</code>	Allows challenge creator to archive their challenge
<code>test_archive_challenge_by_non_creator</code>	Prevents non-creators from archiving (403 Forbidden)
<code>test_archive_already_archived</code>	Ensures idempotent archiving (subsequent archive calls return success)

10. Exercises List Tests (**TestExercisesList**)

Test Name	Description
<code>test_list_exercises</code>	Verifies the exercise catalog returns at least the 3 seeded exercises

Test Environment

Component	Configuration
Database	SQLite in-memory (per test isolation)
Authentication	JWT tokens generated per test
Seed Data	3 exercises: Squats, Pushups, Plank
Test Runner	pytest 9.1.1
HTTP Client	FastAPI TestClient (httpx)

Coverage Summary

Category	Tests	Status
Challenge Creation	9	All Passing
Challenge Detail	3	All Passing
Challenge Edit	3	All Passing
Join/Leave	6	All Passing
Session Submission	6	All Passing
Leaderboard	2	All Passing
Presets	1	All Passing
Me Routes	3	All Passing
Archive	3	All Passing
Exercises List	1	All Passing
Total	37	37/37 Passing

Running the Tests

SHELL

```
# Run all tests with verbose output
pytest tests/test_challenges.py -v
```

```
# Run specific test class
pytest tests/test_challenges.py::TestSessionSubmission -v
```

```
# Run with coverage report
pytest tests/test_challenges.py --cov=app
```

```
# Run failing tests only
pytest tests/test_challenges.py --lf
```

Notes for Developers

1. **Database Isolation:** Each test runs in a fresh database with seeded exercises
2. **Date Handling:** All creation tests use tomorrow's date to bypass validation
3. **Authentication:** Each test generates fresh JWT tokens
4. **Session Accumulation:** Multiple sessions in a day sum clean_reps until goal is met
5. **Streak Logic:** User streak increments only once per calendar day